

MEDIATEK

MT8676 Hypervisor Suspend & Resume User Manual

Version: 1.1
Release date: 2024-11-13

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-10-28	Xiaojun Zheng	Official release
1.1	2024-11-13	Xiaojun Zheng	Added L+L+A

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

List of Tables 3

1 Suspend/Resume 4

1.1 Overview 4

1.2 Architecture/Process Overview 4

1.2.1 Hypervisor Suspend Flow 4

1.2.2 Hypervisor Resume Flow 6

1.3 Frequently Asked Questions/Troubleshooting 7

1.3.1 How to Determine if the System Has Successfully Entered Sleep Mode 7

1.3.2 How to Identify Wakeup Sources 8

1.3.3 Wakeup Source Analysis 8

1.3.4 How to Analyze Issues with Failing to Enter Sleep Mode 9

1.3.5 How to Analyze High Power Consumption During Sleep Mode 10

1.3.6 Hypervisor Debug Command 10

Exhibit 1 Terms and Conditions 12

List of Figures

Figure 1-1. Hypervisor suspend flow 5

Figure 1-2. Hypervisor resume flow 6

Figure 1-3. Wakelock dump 10

List of Tables

Table 1-1. MT8676 wakeup source list 8

1 Suspend/Resume

1.1 Overview

This chapter mainly introduces the MT8676 suspend/resume process and common troubleshooting methods.

On the MT8676 platform, suspend refers to suspend to RAM, a state where all devices enter a low-power mode, with only RAM self-refresh remaining.

Suspend Power State: The entire system is in standby, with very low power consumption.

- Tasks “frozen”
- External devices/internal modules powered off or in low-power mode
- System PLL/clock close
- ARM off
- DRAM self-refresh
- PMIC in low-power mode
- Vcore off
- SPM running
- Wait for HW wakeup event

In the MT8676 Hypervisor VM set (L+L+A), both Host OS SoS and Guest OS Tbox are Yocto, while Guest OS IVI is Android. System-level suspend is initiated by Host OS, which triggers Guest OS to follow the suspend flow. Once all the Guest OS complete their suspend process, then Host OS can enter suspend mode. After Host OS completes its suspend, the entire system can enter a deep sleep mode for power saving.

1.2 Architecture/Process Overview

1.2.1 Hypervisor Suspend Flow

The framework diagram for Hypervisor Suspend is shown in [Figure 1-1](#) (taking EINT trigger system entry and exit from sleep as an example).

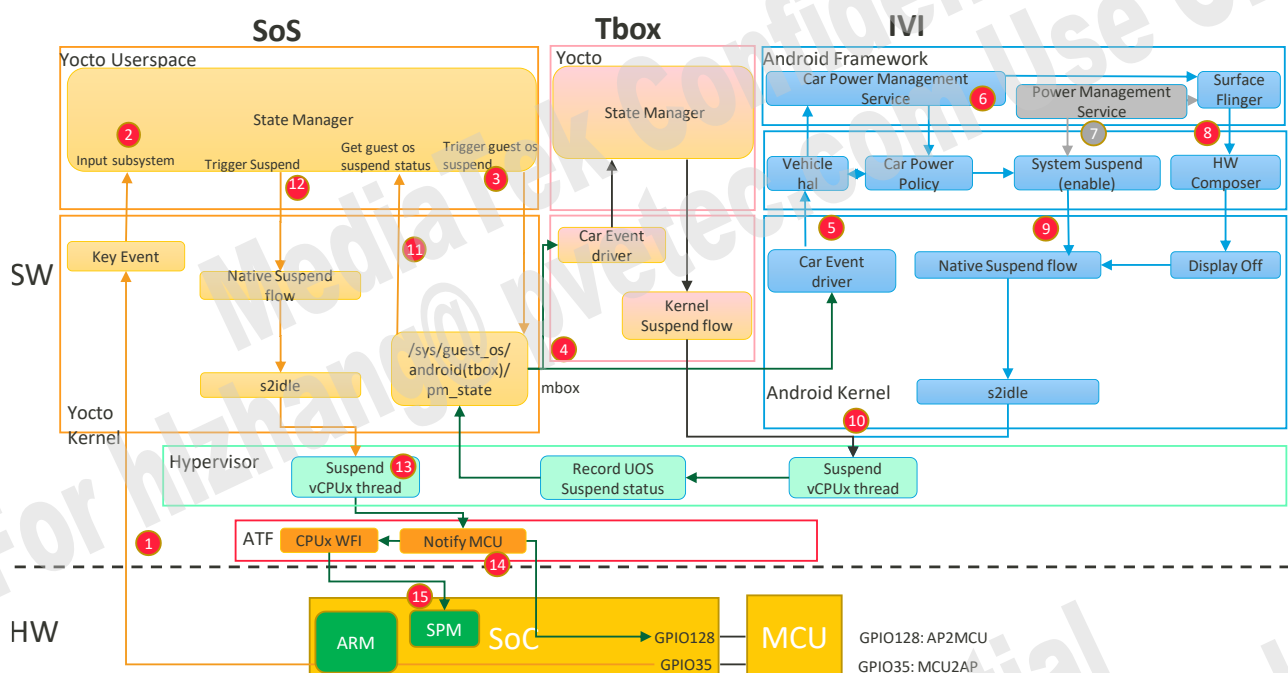


Figure 1-1. Hypervisor suspend flow

Main flow description:

- The platform's entry and exit from sleep mode is primarily initiated by the MCU. The MCU controls the SoC's sleep state through the status of GPIO35 (MCU2AP). When GPIO35 is pulled low, it triggers an interrupt, which is distributed by the Hypervisor to the SoS for processing. Upon receiving the interrupt, the SoS Kernel reports a Key Event.
- In SoS UserSpace, MediaTek has added a control logic called SleepManager (State Manager) for managing the power state of the Guest OS. This logic is primarily implemented by the SoC obtaining key events through the Input Subsystem.
- Upon detecting a sleep event, SoS UserSpace writes a powerkey event to `/sys/guest_os/android(tbox)/pm_state`, initiating a sleep request to the two Guest OS (Android/Yocto).
- SoS Kernel Driver notifies Suspend request via Mailbox to the Car Event Driver of the two Guest OS.
- In the Car Event Driver, a virtual Power Key (Keyevent=87) is reported to trigger the complete sleep process on the Android side.
- In Guest OS IVI, since the Vehicle HAL does not have an existing interface to connect with the Car EVENT Driver, input subsystem support is added here to wait for the virtual Power Key sent by the Kernel. Upon receiving it, VHAL notifies the Car Power Management Service to start the sleep process.
- The Power Management Service still listens for the Power Key Event (Keyevent=116), bypassing the PMS flow.
- The display-related hardware is turned off.
- CPMS triggers the suspend flow without a wakelock check, triggering the Android Kernel Suspend. The Kernel Suspend Flow is the same as that in a native single system. According to the execution order, it sequentially calls the `Prepare/Suspend/Suspend_late` callbacks registered by the devices, completing the necessary preparations for each device driver during the suspend phase.
- After each device enters sleep mode, an `s2idle` SMC command is sent to the Arm Trusted Firmware (ATF) to shut down the CPU core. This command will be intercepted by the Hypervisor, which will then suspend the corresponding

vCPU thread. Once Android's sleep process is complete, the Hypervisor will update the recorded Android state from running to suspend, making it convenient for the Host OS to query the status through a node.

11. After triggering Guest OS sleep, the Host OS side blocks and waits until it retrieves the suspend success of the two Guest OS from the node `/sys/guest_os/android(tbox)/pm_state`.
12. After all Guest OS suspend, the Host OS reports key code 87 to trigger the display-related shutdown. Then, it checks the Wakelock status. If there is no any lock, it will triggers the Host OS Kernel Suspend Flow (`echo mem > /sys/power/state`).
13. Upon executing the Host OS Kernel Suspend and completing the Device Suspend, an `s2idle` SMC command is sent to the ATF to shut down the CPU core. This command will be intercepted by the Hypervisor, which will then suspend the corresponding vCPU thread. Afterward, the virtual machine performs a backup, and the Hypervisor sends the `s2idle` command to the ATF again to actually shut down the physical CPU.
14. In the ATF, before CPU0 executes the Wait For Interrupt (WFI) instruction, GPIO128 (AP2MCU) is pulled low to notify the MCU that the SoC has completed the Host OS and Guest OS Suspend Flow.
15. Once the Arm-related hardware is shut down, the SPM (System Power Management MCU) takes over the system's resources. It controls the DRAM to enter self-refresh mode, shuts down the 26M clock and VCORE, and notifies the PMIC to enter Low Power Mode. At this point, the entire sleep process is complete, and the system is in a low-power state, waiting for a wake-up event to occur.

1.2.2 Hypervisor Resume Flow

"Resume Flow" is the opposite of "Suspend Flow", referring to Figure 1-2:

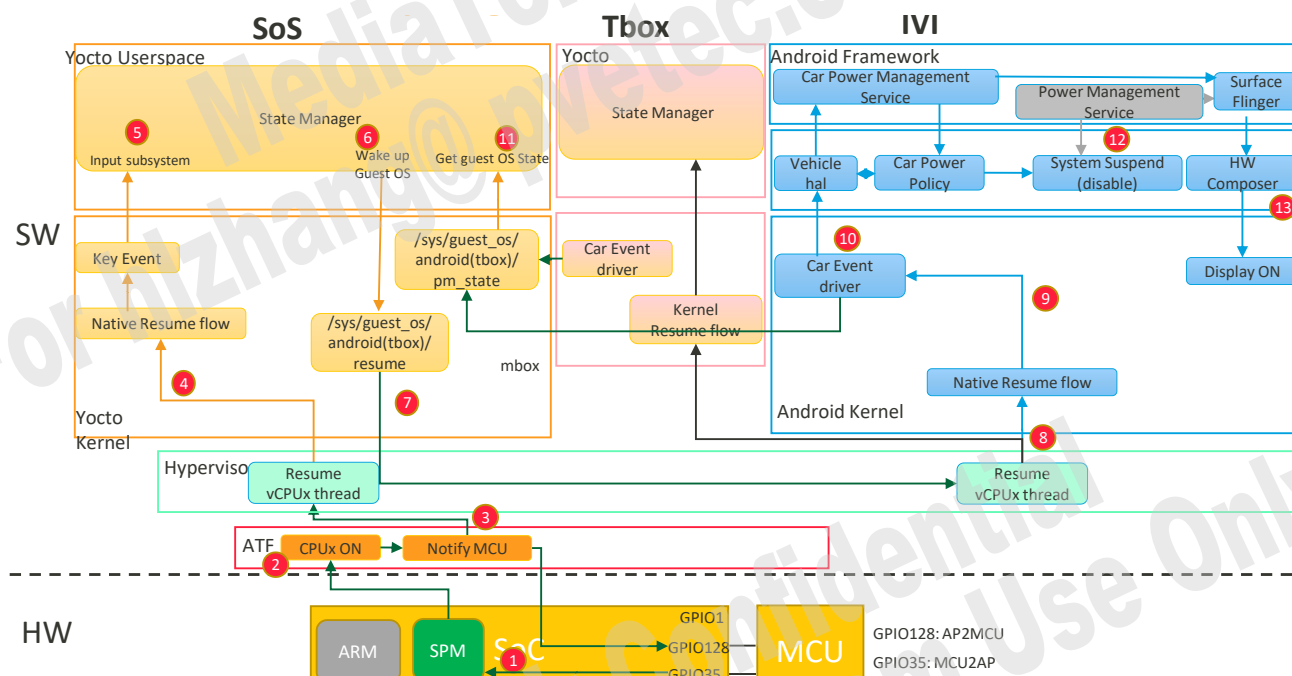


Figure 1-2. Hypervisor resume flow

Main process:

1. After the MCU receives a wake-up event, it pulls up GPIO35 (MCU2AP). The SPM receives an External Interrupt (EINT) and sequentially activates the released resources, such as notifying the PMIC to exit Low Power Mode, turning on VCORE and 26M, and sending a command to notify DRAM to exit self-refresh.
2. Power on CPU0, execute the CPU Resume process, then jump to the address set before Suspend, and start the system Resume process.
3. Pull up GPIO128 to send a message to the MCU indicating that the SoC has woken up. (The timing of notifying the MCU can be adjusted as needed, such as waiting for Android Resume to complete before pulling up GPIO128).
4. After the vCPUx thread in the Hypervisor is restored, it jumps to the Host OS Linux Kernel and then executes the Native Kernel Resume Flow, reporting the wake-up event through a Key Event.
5. The SleepManager in SoS will decide whether to go back to sleep or continue the wake-up flow based on the event reported by the Input subsystem. For example, pulling up GPIO35 by the MCU will continue the wake-up flow.
6. If it is a Key Event that can wake up the entire system, keyevent 87 is reported to resume Weston, and a wakelock is held to prevent the system from going back to sleep.
7. Write to the node `/sys/guest_os/android(tbox)/resume` to trigger Guest OS wake-up.
8. After receiving it, the Hypervisor resumes the vCPU0 thread.
9. Enable Non-Boot CPUs, resume vCPU1~7 threads, and start the IVI and Tbox Kernel Resume Flow.
10. Based on the CPMS architecture, there is no need to determine the keyevent; by default, all events can trigger the entire system to resume.
11. On the Host OS side, the status of the two Guest OS (Android/Tbox) can be queried through the node `/sys/guest_os/android(tbox)/pm_state` using the Mailbox mechanism. If the Guest OS kernel Resume is completed, the related status will be updated to Running.
12. On the android side, after receiving the wake-up event, the Car Power Management Service will disable System Suspend to prevent accidental sleep triggers. On the Tbox side, the SleepManager actively holds the wakelock.
13. Notify the display-related modules to turn on. At this point, the entire wake-up process is complete, and human-machine interaction can begin.

1.3 Frequently Asked Questions/Troubleshooting

1.3.1 How to Determine if the System Has Successfully Entered Sleep Mode

Screen-off might only indicate that the system has entered light sleep mode, which does not necessarily mean that it has successfully entered deep sleep mode. To determine if the system has successfully entered suspend mode, you need to check the kernel log. If the system has successfully entered suspend mode, the kernel log will stop printing.

To enable more detailed debug log information for determining if the system has successfully entered suspend mode, you need to run the following commands.

```
adb shell "echo 8 8 8 8 > /proc/sys/kernel/printk"
adb shell "echo 1 > /sys/module/kernel/parameters/initcall_debug"
adb shell "echo 1 > /proc/mtprintk"
```

In the kernel log, the following keywords indicate the progress of the suspend process.

1. “PM: Syncing filesystems ...”:

This indicates that the kernel has started the suspend process.

2. “suspend of devices complete after xxx msecs”:

This indicates that the device suspend phase has been completed.

3. “late suspend of devices complete after xxx msecs”:

This indicates that the late suspend phase for devices has been completed.

4. “noirq suspend of devices complete after xxx msecs”:

This indicates that the noirq (no interrupt request) suspend phase for devices has been completed.

5. “suspend enter”:

This indicates that the suspend process is complete and the system has entered the suspend state.

1.3.2 How to Identify Wakeup Sources

Search for the keyword “suspend wake up by” in the kernel log to identify the wakeup source.

e.g.

Pwrkey Wakeup (and other EINT Wakeups):

```
[SPM] suspend wake up by R12_EINT_EVENT_B, timer_out = 207308
```

To determine which specific EINT caused the wakeup, first look for the log entry:

```
EINT xxx is pending
```

Then, use the following command to see which interrupt corresponds to xxx:

```
cat /proc/interrupts
```

Modem-Related Wakeup:

```
[SPM] suspend wake up by R12_CCIF0_EVENT_B, timer_out = 1825253
```

Timer (PCM_Timer) Wakeup:

```
[SPM] suspend wake up by PCM_TIMER, timer_out = 65612
```

1.3.3 Wakeup Source Analysis

Table 1-1 is a list of wakeup sources supported by the MT8676 platform.

Table 1-1. MT8676 wakeup source list

Name	Control Bit	Description	Can it be Disabled
R12_PCM_TIMER	0	Timer event for its timeout setting	Y

Name	Control Bit	Description	Can it be Disabled
R12_KP_IRQ_B	2	Keypad pressed/released action been detected	Y
R12_APWDT_EVENT_B	3	RGU wakes up CPU in suspend mode.	Y
R12_APXGPT1_EVENT_B	4	AP GPT timer timeout event	Y
R12_CONN2AP_SPM_WAKEUP_B	5	Connectivity IC (Wi-Fi/BT/GPS) event	Y
R12_EINT_EVENT_B	6	EINT event	N
R12_CONN_WDT_IRQ_B	7	Connectivity IC watchdog timeout event	Y
R12_CCIF0_EVENT_B	8	MD to AP CCIF wakeup event	Y
R12_CCIF1_EVENT_B	9	MD to AP CCIF wakeup event	Y
R12_SSPM2SPM_WAKEUP_B	10	SSPM event	Y
R12_SCP2SPM_WAKEUP_B	11	SCP sensor event	N
R12_ADSP2SPM_WAKEUP_B	12	ADSP event	Y
R12_USBX_CDSC_B	14	USB event	Y
R12_USBX_POWERDOWN_B	15	USB remote wakeup	Y
R12_SYS_TIMER_EVENT_B	18	System timer	N
R12_EINT_EVENT_SECURE_B	19	EINT event	N
R12_SCP_CIRQ_IRQ_B	22	SCP_CIRQ wakeup event	Y
R12_MD2AP_PEER_EVENT_B	23	MD event	Y
R12_MD1_WDT_B	25	MD1 watchdog timeout	Y
R12_REG_CPU_WAKEUP_B	28	Internal Wakeup	N
R12_APUSYS_WAKE_HOST_B	29	APUSYS event	Y
R12_PCIE_WAKEUP_EVENT_B	30	PCIe event	Y
R12_MSDC_WAKEUP_EVENT_B	31	MSDC event	Y

The wakeup sources in the list can be disabled through the following way, but only the wakeup sources marked as “Y” in the “Can it be Disabled” column can be disabled.

You can quickly disable the desired wakeup sources using the system-provided debug command.

1. Use the following command to read the current system wakeup source settings.

```
cat /proc/mtk_lpm/power/suspend_ctrl | grep -i wake_src
```

2. In [Table 1-1](#), find the control bit of the wakeup source you want to disable in the “Control Bit” column. Then, set the corresponding bit of the read wake_src to 0, and finally use the following command to write the modified wake_src back into the system (do not modify the control bits of wakeup sources you do not want to disable):

```
echo wake_src 0x***** > /proc/mtk_lpm/power/suspend_ctrl
```

3. If you want to re-enable a wakeup source, set the corresponding control bit back to 1.

1.3.4 How to Analyze Issues with Failing to Enter Sleep Mode

If the system fails to enter sleep mode after the screen turns off, you can use the following commands to determine which wakelock is preventing the system from entering sleep mode.

- Check which modules in the kernel are holding wakelocks

```
cat /sys/kernel/debug/wakeup_sources
```

Observe the 5th column “active_since” in the output. The number that is non-zero and continuously increasing indicates the wakelock that is preventing the system from entering standby mode.

For example, in [Figure 1-3](#), the USB wakelock is blocking the system from entering standby mode.

```
spm8675p1_64:/ # cat /sys/kernel/debug/wakeup_sources
```

name	active_count	event_count	wakeup_count	expire_count	active_since	total_time	max_time	last_change	prevent_suspend_time
event2	0	0	0	0	0	0	0	0	0
event1	8	8	0	0	0	203	145	41989	0
event3	0	0	0	0	0	0	0	0	0
event0	0	0	0	0	0	0	0	0	0
	20	20	0	0	0	4	1	130596	0
inotify	2	2	0	0	0	0	0	31562	0
eventpoll	20	20	0	0	0	0	0	130596	0
(timerfd)	18	18	9	0	0	1223	151	41937	0
(timerfd)	0	0	0	0	0	0	0	0	0
(timerfd)	18	18	4	0	0	575	151	121306	0
(timerfd)	20	20	2	0	0	284	131	37344	0
(timerfd)	0	0	0	0	0	0	0	0	0
(timerfd)	0	0	0	0	0	0	0	0	0
eventpoll	44	56	0	0	0	0	0	121306	0
spswakelock	0	0	0	0	0	0	0	0	0
ot_drv	0	0	0	0	0	0	0	0	0
pmrFuncCtrl	3	3	0	0	0	2872	1456	43697	0
MT662x	7	7	0	0	0	374	72	52259	0
vpul	0	0	0	0	0	0	0	0	0
vpul0	0	0	0	0	0	0	0	0	0
msub-hdrc	2	2	0	0	107302	134244	107302	41049	0
mdla	0	0	0	0	0	0	0	0	0
sibc	0	0	0	0	0	0	0	0	0

Figure 1-3. Wakelock dump

The command needs to be entered via UART because plugging in a USB device can prevent the system from entering standby mode.

1.3.5 How to Analyze High Power Consumption during Sleep Mode

- First check for frequent wakeup issues. If there are frequent wakeup issues, please refer to [Section 1.3.3](#) to identify the wakeup sources.
- If the system successfully enters sleep mode but the power consumption is still high, first check for any peripherals that may not be properly powered down.
- If the high power consumption is traced to the MediaTek SoC, please provide logs to MediaTek for further analysis.

1.3.6 Hypervisor Debug Command

- Check the status of the IVI(Android) OS on the SoS side, while “running” indicates that Android is currently running, while “suspend” indicates that Android has been suspended.

```
cat /sys/guest_os/android/pm_state
```

- Trigger the IVI(Android) OS to enter suspend mode on the SoS side.

```
echo powerkey > /sys/guest_os/android/pm_state
```

- Trigger the IVI(Android) OS to resume on the SoS side.

```
echo 0 > /sys/guest_os/android/resume
```

4. Check the status of the Tbox OS on the SoS side, while “running” indicates that Tbox is currently running, while “suspend” indicates that Tbox has been suspended.

```
cat /sys/guest_os/tbox/pm_state
```

5. Trigger the Tbox OS to enter suspend mode on the SoS side.

```
echo powerkey > /sys/guest_os/tbox/pm_state
```

6. Trigger the Tbox OS to resume on the SoS side.

```
echo 1 > /sys/guest_os/tbox/resume
```

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.